QAgent : agent de test automatique multi-niveaux

> Agent open-source en **Java 21** / **Spring Boot** qui **conçoit, génère, exécute, répare et rapporte** les tests d'une application : unitaires, intégration, API (en remplacement de Postman) et contrat.
> **Statut :** cadrage · **Dernière mise à jour :** 21/10/2026 Ce document est la **référence unique** du projet. Toute décision d'architecture qui le contredit doit d'abord le modifier (voir §15 Gouvernance).

Sommaire

1. Vision et problème
2. Périmètre
3. Principes d'architecture
4. **Shared Kernel : définition de référence**
5. Vue d'ensemble des modules
6. Détail des modules
7. Règles de dépendance entre modules
8. Catalogue des événements
9. Pipeline d'un run
10. Le problème de l'oracle de test
11. Environnements, données et sécurité
12. Rapport final
13. Stack technique
14. Structure du dépôt
15. Gouvernance et conventions
16. Stratégie de test de QAgent lui-même
17. Risques et parades
18. Feuille de route
19. Exemples d'usage
20. Décisions ouvertes
21. Glossaire

---

1. Vision et problème

Aujourd'hui, les équipes testent « à l'ancienne » :

- tests unitaires écrits à la main, souvent insuffisants ;
- API testées manuellement avec Postman (collections longues à maintenir, peu automatisées) ;
- aucun rapport unifié de la qualité réelle des tests.

QAgent automatise la chaîne complète. À partir d'une application (code + spécification API), il :

1. analyse le code et découvre les API ;
2. planifie les tests à écrire ;
3. génère les tests (JUnit 5, Mockito, AssertJ, REST Assured) ;
4. les compile et les exécute en sandbox ;
5. corrige les tests défaillants et **signale** les vrais bugs de l'application ;
6. mesure la couverture et la qualité réelle (mutation testing) ;
7. produit un **rapport final** et, au choix, une Pull Request.

Objectifs mesurables (KPI)

KPI	Cible
Couverture lignes/branches (code ciblé)	⩾ 80 %
Couverture API	100 % des endpoints, au moins un cas par code de statut documenté
Tests générés qui passent après réparation	⩾ 95 %
Mutation score (code ciblé)	⩾ 60 %
Durée d'un run complet (projet moyen)	< 30 min

2. Périmètre

Niveau	Ce que l'agent produit	Outils générés
Unitaire	Tests des classes et méthodes	JUnit 5, Mockito, AssertJ
Intégration	Tests entre couches (service + repository + base de données)	Spring Boot Test, Testcontainers
API (remplace Postman)	Scénarios par endpoint : succès, validation, erreurs, authentification, enchaînements	REST Assured / MockMvc
Contrat	Vérification que l'API respecte son OpenAPI	Validateurs OpenAPI

Hors périmètre : tests fonctionnels/E2E (UI), tests de performance, tests de sécurité, correction automatique du code métier, langages autres que Java (extensible plus tard).

Sources d'entrée de l'agent

1. **Code source** (Git ou dossier local).
2. **Spécification OpenAPI/Swagger** (ou reconstruite depuis les annotations Spring MVC).
3. **Collections Postman existantes** (import : requêtes, variables, scripts simples).
4. **Documentation / user stories** (optionnel), pour déduire des règles métier.
5. **Environnement cible** : URL de test, identifiants de test, base de données de test.

Acteurs

Acteur	Cas d'usage
Développeur	Lance l'agent sur une classe, un module ou un endpoint
Testeur/QA	Importe ses collections Postman, relit et valide les scénarios
Tech lead	Fixe les seuils (quality gates), lit le rapport
CI/CD	Déclenche l'agent à chaque Pull Request, sur le périmètre modifié
Admin	Configure LLM, environnements, quotas, confidentialité

---

3. Principes d'architecture

3.1 Application modulaire + hexagonale

- **Spring Boot modulaire** : Maven multi-module + Spring Modulith.
- **Chaque module suit l'architecture hexagonale** (ports et adapters).
- Les modules communiquent **uniquement** par leurs **ports in** publics et par des **événements**. Jamais d'accès au domaine interne ou aux adapters d'un autre module.

3.2 Structure hexagonale type d'un module

```
<module>/src/main/java/com/qagent/<module>/
├── domain/
│   ├── model/        entités, objets-valeur, agrégats du module
│   ├── service/      règles métier pures
│   └── event/        événements émis par le module
├── application/
│   ├── port/in/      cas d'usage exposés (interfaces) : API publique du
module
│   ├── port/out/     besoins du module (interfaces) : implémentés par les
adapters
│   └── service/      implémentation des ports in (orchestration du domaine)
└── adapter/
    ├── in/           REST, listeners d'événements, CLI
    └── out/          LLM, fichiers, JPA, Git, Docker, outils externes…
```

3.3 Règles de couche (vérifiées automatiquement)

Couche	Peut dépendre de	Ne doit jamais contenir
`domain`	`shared-kernel` uniquement	Spring, JPA, HTTP, JSON, I/O
`application`	`domain`, `shared-kernel`	Annotations d'infrastructure, accès direct aux outils externes
`adapter`	`application`, `domain`, `shared-kernel`, bibliothèques externes	Règles métier

Ces règles sont vérifiées par **ArchUnit** et `ApplicationModules.verify()` (Spring Modulith). **Le build échoue en cas de violation.**

3.4 Décision clé : que génère-t-on pour les tests API ?

Deux représentations complémentaires :

1. Un **modèle de scénario neutre** (JSON/DSL interne) sur lequel l'agent raisonne.
2. Du **vrai code Java** (REST Assured + JUnit 5) généré depuis ce modèle, versionné dans le dépôt, exécutable en CI **sans l'agent**.

Le même modèle alimente l'**export Postman/Newman**, ce qui facilite l'adoption par les équipes qui restent sur Postman.

---

4. Shared Kernel : définition de référence

> Le `shared-kernel` est défini **avant tout autre module**. C'est le premier module créé et le plus strictement gouverné : une mauvaise décision ici se propage partout.

4.1 Qu'est-ce que c'est ?

Le **vocabulaire minimal commun** à tous les modules : identifiants typés, énumérations partagées, petits objets-valeur, contrat des événements et hiérarchie d'erreurs de base. Ce n'est **pas** un module métier, ni un « utils ».

- Artefact Maven : `qagent-shared-kernel`
- Package racine : `com.qagent.shared`
- Dépendances externes : **aucune** (JDK 21 uniquement)

4.2 Contenu autorisé

Catégorie	Contenu	Package
Identifiants typés	`RunId`, `ProjectId`, `TargetId`, `TestId`, `EndpointId`	`shared.id`
Énumérations partagées	`TestLevel`, `TestStatus`, `FailureCategory`, `HttpMethod`	`shared.model`
Objets-valeur	`QualifiedName` (nom Java complet), `SourceLocation`, `Percentage`, `Confidence`	`shared.model`
Contrat d'événement	`DomainEvent` (interface)	`shared.event`
Erreurs de base	`QAgentException`, `NotFoundException`, `InvalidInputException`, `ExternalSystemException`, `ErrorCode`	`shared.error`
Garde-fous	`Guard` (vérification d'invariants : non nul, non vide, bornes)	`shared.error`

4.3 Contenu de référence (code)

java

```java
package com.qagent.shared.id;

import java.util.UUID;

/** Identifiant d'un run. Même modèle pour ProjectId, TargetId, TestId. */
public record RunId(UUID value) {
    public RunId { Guard.notNull(value, "RunId.value"); }
    public static RunId newId() { return new RunId(UUID.randomUUID()); }
    @Override public String toString() { return value.toString(); }
}

/** Un endpoint est identifié par sa méthode HTTP et son chemin normalisé. */
public record EndpointId(HttpMethod method, String path) {
    public EndpointId {
        Guard.notNull(method, "method");
        Guard.notBlank(path, "path");
    }
}
```

java

```java
package com.qagent.shared.model;

public enum TestLevel { UNIT, INTEGRATION, API, CONTRACT }

public enum TestStatus {
    PLANNED, GENERATED, COMPILED, PASSED, FAILED, QUARANTINED, DISCARDED
}

/** Résultat de la classification d'un échec (module failure-analysis). */
public enum FailureCategory {
    TEST_ERROR,            // le test est faux : à réparer
    ENVIRONMENT_OR_FLAKY,  // instabilité : relance ou quarantaine
    SUSPECTED_APP_BUG      // l'application diverge de sa spec : à signaler
}

public enum HttpMethod { GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS }

public record QualifiedName(String value) {          // ex. com.acme.order.Order
    public QualifiedName { Guard.notBlank(value, "QualifiedName"); }
}

public record SourceLocation(String file, int line, int column) { }

public record Percentage(double value) {              // borné à [0, 100]
    public Percentage { Guard.inRange(value, 0, 100, "Percentage"); }
}

public record Confidence(double value) {              // borné à [0, 1]
    public Confidence { Guard.inRange(value, 0, 1, "Confidence"); }
}
```

java

```java
package com.qagent.shared.event;

import com.qagent.shared.id.RunId;
import java.time.Instant;
import java.util.UUID;

/** Tout événement de domaine du système implémente ce contrat. */
public interface DomainEvent {
    UUID eventId();
    RunId runId();          // tout événement est rattaché à un run
    Instant occurredAt();
}
```

```java
package com.qagent.shared.error;

/** Racine de toutes les erreurs métier. Sans dépendance de framework. */
public abstract class QAgentException extends RuntimeException {
    private final ErrorCode code;
    protected QAgentException(ErrorCode code, String message) { super(message);
    protected QAgentException(ErrorCode code, String message, Throwable cause)
    public ErrorCode code() { return code; }
}
```

4.4 Règles du shared-kernel

#	Règle	Pourquoi
SK-1	**Zéro dépendance externe** : JDK uniquement. Pas de Spring, JPA, Jackson, Lombok.	Le domaine de chaque module dépend du kernel ; une dépendance ici contaminerait tout.
SK-2	**Immuable** : records et enums uniquement (pas d'état modifiable).	Sûr pour le parallélisme et les événements.
SK-3	**Aucune logique métier.** Seulement la validation des invariants du type lui-même.	Le métier appartient aux modules.
SK-4	**Règle des 3 modules** : un concept n'entre dans le kernel que s'il est utilisé par **au moins 3 modules** et que sa sémantique est identique partout.	Évite le kernel « fourre-tout » qui recrée un monolithe couplé.
SK-5	**Aucune dépendance vers un module.** Le kernel ne connaît personne.	Sens unique : modules → kernel.
SK-6	**Taille plafonnée** : environ 40 types au maximum. Revue à chaque fin de phase.	Un kernel qui grossit est un signal d'alerte de conception.
SK-7	**Changement contrôlé** : ajout compatible = PR + 1 relecture ; renommage ou suppression = ADR + 2 relectures + version majeure.	Tous les modules sont impactés.
SK-8	**Pas de sérialisation intégrée** : aucune annotation JSON/JPA. Les adapters s'occupent de la conversion.	Garde le kernel indépendant de l'infrastructure.

4.5 Ce qui n'y va PAS (exemples)

Concept	Où le mettre	Pourquoi pas dans le kernel
`RunStatus`, `RunConfiguration`	`workflow-orchestrator`	Concerne un seul module
`ApiEndpoint` (avec schémas, réponses)	`api-discovery`	Modèle riche propre à un module ; les autres reçoivent une vue adaptée. Seul `EndpointId` est partagé
`CoverageReport`	`quality-metrics`	Propre à un module ; `reporting` le lit via un port
`ModelProfile`, `LlmRequest`	`llm-gateway`	Détail d'implémentation du gateway
Utilitaires de fichiers, chaînes, JSON	Dans l'adapter qui en a besoin	« Utils » = fourre-tout

4.6 Test d'architecture du kernel (obligatoire)

java

```java
@AnalyzeClasses(packages = "com.qagent.shared")
class SharedKernelArchitectureTest {

    @ArchTest
    static final ArchRule no_external_dependencies =
        noClasses().that().resideInAPackage("com.qagent.shared..")
            .should().dependOnClassesThat().resideOutsideOfPackages(
                "com.qagent.shared..", "java..");

    @ArchTest
    static final ArchRule no_framework_annotations =
        noClasses().that().resideInAPackage("com.qagent.shared..")
            .should().beAnnotatedWith("org.springframework..")
            .orShould().beAnnotatedWith("jakarta..");
}
```

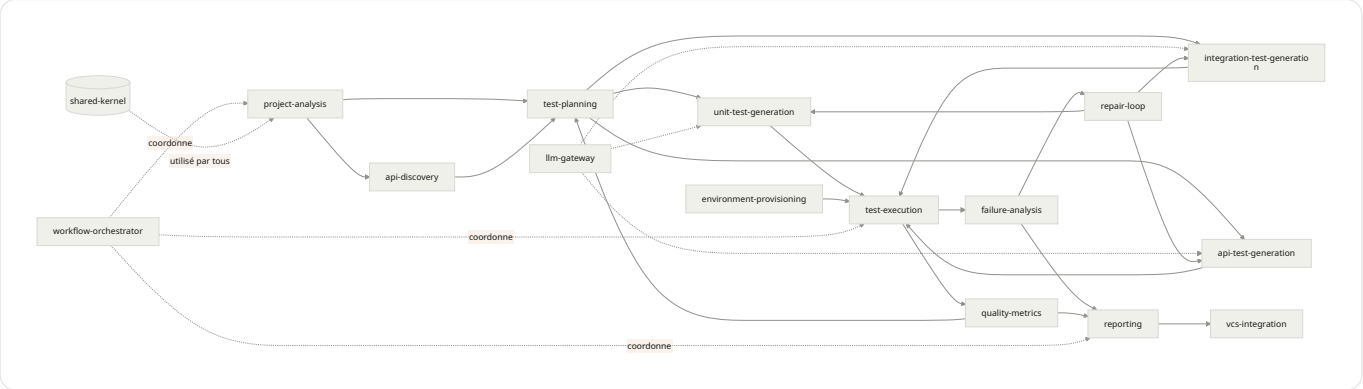
À cela s'ajoute, dans chaque autre module, une règle : *« les classes du `domain` ne dépendent que d'elles-mêmes, de `com.qagent.shared..` et de `java..` ».*

---

5. Vue d'ensemble des modules

17 modules en 7 familles :

Famille	Modules	Question
Fondation	`shared-kernel`	Quel vocabulaire commun ?
Compréhension	`project-analysis`, `api-discovery`	Que contient et qu'expose l'application ?
Décision	`test-planning`	Quoi tester, à quel niveau, dans quel ordre ?
Génération	`llm-gateway`, `unit-test-generation`, `integration-test-generation`, `api-test-generation`	Comment produire les tests ?
Exécution	`environment-provisioning`, `test-execution`	Où et comment les exécuter ?
Évaluation	`failure-analysis`, `repair-loop`, `quality-metrics`	Les tests sont-ils bons ? Que faire des échecs ?
Pilotage et sortie	`workflow-orchestrator`, `reporting`, `vcs-integration`, `bootstrap`	Qui coordonne, présente, livre ?



6. Détail des modules

> Format de chaque fiche : **rôle**, **responsabilités**, **domaine**, **ports in** (API publique), **ports out** (besoins), **adapters**, **événements**, **points d'attention**.

6.1 `shared-kernel`

Voir §4. Pas de ports, pas d'adapters : bibliothèque pure.

6.2 `project-analysis`

Rôle : lire et comprendre le code Java du projet cible.

Responsabilités

- Parcourir les sources et construire un modèle : packages, classes, méthodes, signatures, annotations (`@Service`, `@RestController`, `@Entity`…).
- Calculer des métriques : complexité cyclomatique, branches, lignes, couplage.
- Construire le **graphe de dépendances** (qui appelle/injecte quoi), indispensable pour savoir quoi mocker.
- Détecter les classes déjà testées et les tests existants.
- Détecter les **fichiers modifiés** (diff Git) pour les runs sur Pull Request.
- Évaluer la **testabilité** (constructeurs complexes, statiques, `new` en dur, singletons).

Domaine : `SourceUnit`, `MethodInfo`, `DependencyGraph`, `ComplexityMetrics`, `TestabilityScore`.

Ports in : `AnalyzeProjectUseCase`, `GetSourceUnitQuery`, `GetDependencyGraphQuery`.

Ports out : `SourceCodeReader`, `AnalysisRepository`, `ChangeDetector`.

Adapters : JavaParser (+ résolution de symboles), système de fichiers, JGit (diff), JPA (cache).

Événement : `ProjectAnalyzed`.

Attention : la résolution de symboles exige le classpath du projet cible, donc un adapter qui résout les dépendances Maven/Gradle.

6.3 `api-discovery`

Rôle : découvrir ce que l'application **expose** comme API et en produire une description unifiée.

Responsabilités

- Lire une spec **OpenAPI/Swagger** (fichier ou URL de l'application en test).
- Sinon, **reconstruire** la description depuis le code (`@RestController`, `@GetMapping`, `@RequestBody`, `@Valid`, validations).
- **Importer des collections Postman** existantes (requêtes, variables, scripts).
- Fusionner les trois sources et signaler les **écarts** (endpoint dans le code mais pas dans la spec, champ obligatoire non documenté…).
- Déduire les **dépendances entre endpoints** (POST avant GET, login avant appels protégés).
- Modéliser la sécurité (JWT, OAuth2, API key) et les rôles requis par endpoint.

Domaine : `ApiEndpoint` (méthode, chemin, paramètres, corps, réponses par code, sécurité), `ApiSchema`, `EndpointDependency`, `SecurityRequirement`, `SpecDiscrepancy`.

Ports in : `DiscoverApiUseCase`, `ImportPostmanCollectionUseCase`, `GetApiCatalogQuery`.

Ports out : `OpenApiLoader`, `SpringMappingScanner`, `PostmanCollectionReader`,

`ApiCatalogRepository`.

Adapters : swagger-parser, scanner basé sur `project-analysis`, lecteur Postman v2.1, JPA.

Événement : `ApiDiscovered`.

Attention : la spec peut être fausse ou périmée. Le module garde la **provenance** de chaque information (spec, code, Postman) pour que `failure-analysis` puisse arbitrer.

6.4 `test-planning`

Rôle : le « cerveau stratégique » : décider **quoi tester, à quel niveau, avec quelle priorité**.

Responsabilités

- Prendre en entrée l'analyse du code, le catalogue d'API et la configuration du run (niveaux, périmètre, seuils).
- Produire une **matrice de tests** : pour chaque classe/méthode/endpoint, les niveaux et types de cas (nominal, limites, erreurs, accès 401/403, validation).
- **Prioriser** : complexité élevée, code non couvert, fichiers modifiés, endpoints critiques.
- Estimer l'effort et le coût (appels LLM) et respecter le **budget** du run.
- Décider d'une **passe supplémentaire** à partir des lacunes de `quality-metrics`.
- Éviter les doublons avec les tests existants.

Domaine : `TestPlan`, `TestTarget` (cible, niveau, priorité, stratégie), `TestCaseSpec`, `Budget`.

Ports in : `CreateTestPlanUseCase`, `RefinePlanFromGapsUseCase`.

Ports out : `AnalysisQuery`, `ApiCatalogQuery`, `PlanRepository`.

Adapters : JPA, adapters de lecture vers `project-analysis` et `api-discovery`.

Événement : `TestPlanCreated`.

Attention : la priorisation est du **domaine pur** (règles testables sans LLM). Un LLM pourra l'affiner plus tard via un port.

6.5 `llm-gateway`

Rôle : porte d'entrée **unique** vers les modèles de langage. Aucun autre module ne sait quel LLM est utilisé.

Responsabilités

- Interface simple : prompt en entrée, réponse (texte ou structurée) en sortie.
- Multi-fournisseurs : Ollama (local), OpenAI, Anthropic, etc.
- **Routage** par type de tâche (petit modèle local pour le simple, plus puissant pour le difficile).
- **Retries, timeouts, limites de débit, quotas, budget** par run.

- **Filtrage des secrets** avant envoi ; politique « local uniquement » pour les projets sensibles.
- **Journalisation** de chaque appel (prompt, réponse, tokens, coût, durée).
- Validation et parsing des **sorties structurées** (JSON, blocs de code).
- Cache optionnel (rend les tests de l'agent déterministes).

Domaine : `LlmRequest`, `LlmResponse`, `ModelProfile`, `UsageRecord`, `PrivacyPolicy`.

Ports in : `CompleteUseCase`, `CompleteStructuredUseCase`.

Ports out : `LlmProvider` (un adapter par fournisseur), `UsageRepository`, `SecretScanner`.

Adapters : Spring AI ou LangChain4j, détecteur de secrets, JPA.

Attention : c'est ici qu'on place un `FakeLlmProvider` déterministe, utilisé par tous les tests de l'agent.

6.6 `unit-test-generation`

Rôle : générer les **tests unitaires** (JUnit 5 + Mockito + AssertJ).

Responsabilités

- Construire un **contexte** par cible : code de la classe, signatures des dépendances à mocker, conventions de test du projet (nommage, assertions, utilitaires).
- Composer les **prompts** par stratégie : nominal, valeurs limites, null/vide, exceptions, branches non couvertes.
- Appeler `llm-gateway`, **parser** la réponse en fichier de test Java.
- Appliquer des **règles de qualité** avant exécution : au moins une assertion par test, pas de `Thread.sleep`, pas de dépendance à l'ordre, pas d'accès réseau, noms lisibles.
- Placer le fichier au bon endroit (`src/test/java/...`, même package).
- **Compléter** un test existant plutôt que l'écraser.
- Régénérer partiellement à la demande de `repair-loop`.

Domaine : `GeneratedUnitTest`, `MockSpec`, `TestStrategy`, `GenerationContext`, `QualityRule`.

Ports in : `GenerateUnitTestsUseCase`, `RegenerateTestUseCase`.

Ports out : `LlmClient`, `TestCodeWriter`, `ProjectConventionsReader`, `GeneratedTestRepository`.

Adapters : client interne vers `llm-gateway`, écriture de fichiers, analyse de conventions (JavaParser).

Événement : `UnitTestsGenerated`.

6.7 `integration-test-generation`

Rôle : générer les tests d'**intégration entre couches** : service + repository + base de données, configuration Spring, sérialisation.

Responsabilités

- Identifier les points d'intégration (repositories JPA, clients HTTP, files de messages, transactions).
- Générer des tests **Spring Boot Test** par tranches (`@DataJpaTest`, `@WebMvcTest`, `@SpringBootTest`) selon le besoin.
- Utiliser **Testcontainers** (vraie base) plutôt que H2 quand le comportement diffère.
- Remplacer les services externes par **WireMock** ou des doubles.
- Préparer les **jeux de données** (fixtures, scripts SQL, builders).
- Vérifier requêtes JPA, mappings, validations, transactions et rollback.

Domaine : `GeneratedIntegrationTest`, `IntegrationSlice` (web, data, full), `FixtureSpec`, `ExternalDependencyStub`.

Ports in : `GenerateIntegrationTestsUseCase`.

Ports out : `LlmClient`, `TestCodeWriter`, `FixtureFactory`, `ProjectConfigReader`.

Adapters : gateway LLM, écriture de fichiers, lecture de config Spring.

Événement : `IntegrationTestsGenerated`.

Attention : dépend fortement de `environment-provisioning` (ce module déclare les conteneurs nécessaires, l'autre les fournit).

6.8 `api-test-generation`

Rôle : le module qui **remplace Postman**. Il génère des scénarios de test d'API.

Responsabilités

- À partir du catalogue d'API, construire des **scénarios** :
  - cas nominal par endpoint ;
  - validation des entrées (champs manquants, types erronés, limites, formats) ;
  - codes d'erreur (400, 404, 409, 422…) ;
  - **autorisations** : sans token (401), mauvais rôle (403), bon rôle (200) ;
  - **enchaînements** : créer, lire, modifier, supprimer, avec capture d'identifiants et de tokens ;
  - **contrat** : la réponse respecte le schéma OpenAPI.
- Produire d'abord un **modèle de scénario neutre**, puis le **traduire en code** REST Assured + JUnit 5.
- Générer les **données de test** (valides, limites, invalides) et leur nettoyage.
- Réutiliser et compléter les scénarios importés de Postman.

- **Exporter** le même modèle en collection Postman/Newman.

Domaine : `ApiScenario`, `ScenarioStep`, `RequestSpec`, `Assertion`, `Extraction` (variables capturées), `TestDataSpec`, `AuthProfile`.

Ports in : `GenerateApiTestsUseCase`, `ExportToPostmanUseCase`.

Ports out : `ApiCatalogQuery`, `LlmClient`, `TestCodeWriter`, `TestDataFactory`, `PostmanExporter`.

Adapters : génération REST Assured, export Postman JSON, gateway LLM.

Événement : `ApiScenariosGenerated`.

Attention : les assertions viennent de la **spec**, pas du comportement observé (voir §10). Le `ScenarioPlanner` du domaine fixe les règles (déterministes) ; le LLM ne fait que remplir valeurs et formulation.

6.9 `environment-provisioning`

Rôle : préparer un **environnement d'exécution propre, isolé et jetable**.

Responsabilités

- Démarrer les dépendances : base de données, broker, services externes simulés (WireMock).
- Démarrer l'**application cible** (build + lancement) sur un port aléatoire.
- Appliquer migrations et jeu de données de base.
- Fournir à `test-execution` les **coordonnées** de l'environnement (URL, identifiants de test).
- **Détruire** l'environnement à la fin ou en cas d'échec, sans résidus.
- Appliquer les **garde-fous** : liste blanche d'URL, interdiction de cibler la production, opérations destructrices autorisées seulement sur environnements « jetables ».
- Gérer plusieurs environnements en parallèle sans collision.

Domaine : `Environment`, `EnvironmentSpec`, `EnvironmentStatus`, `TargetPolicy`.

Ports in : `ProvisionEnvironmentUseCase`, `DestroyEnvironmentUseCase`.

Ports out : `ContainerRuntime`, `ApplicationLauncher`, `DatabaseSeeder`, `PortAllocator`.

Adapters : Testcontainers / API Docker, lanceur Maven/Gradle/Jar, Flyway/SQL.

Événements : `EnvironmentReady`, `EnvironmentDestroyed`.

6.10 `test-execution`

Rôle : compiler et exécuter les tests générés, de façon sûre.

Responsabilités

- Copier les tests dans une **copie de travail** du projet (jamais directement dans le dépôt de l'utilisateur).
- **Compiler** et capturer les erreurs de compilation de façon structurée (fichier, ligne, message).
- **Exécuter** via Maven/Gradle (Tooling API) en **sandbox** : limites CPU/RAM/temps, réseau restreint, système de fichiers limité.
- Paralléliser par lots, gérer les **timeouts**.
- Collecter des résultats structurés (JUnit XML) : réussis, échoués, ignorés, durées, traces, journaux.
- Pour l'API : capturer les **requêtes et réponses** complètes de chaque échec.
- **Relancer** sélectivement un sous-ensemble (après réparation).

Domaine : `ExecutionRequest`, `ExecutionResult`, `CompilationError`, `TestOutcome`, `ResourceLimits`, `HttpExchange`.

Ports in : `ExecuteTestsUseCase`, `RerunTestsUseCase`.

Ports out : `BuildToolRunner`, `Sandbox`, `ResultParser`, `WorkspaceManager`.

Adapters : Maven/Gradle Tooling API, exécution en conteneur Docker, parseur JUnit XML.

Événement : `TestsExecuted`.

Attention : module le plus sensible en sécurité : il exécute du code généré par un LLM à partir de code potentiellement non fiable.

6.11 `quality-metrics`

Rôle : mesurer la **vraie valeur** des tests, pas seulement leur nombre.

Responsabilités

- **Couverture de code** (lignes, branches, méthodes) via JaCoCo, par classe/package/module, comparaison **avant/après**.
- **Couverture API** : matrice endpoint × méthode × code de statut (testé / non testé).
- **Mutation testing** (PIT) : mutation score et **mutants survivants**.
- **Détection de tests flaky** : exécutions répétées, repérage des résultats instables.
- Évaluer les **quality gates** (ex. couverture ⩾ 80 %, couverture API = 100 %).
- Identifier les **lacunes** à transmettre à `test-planning` pour une passe ciblée.

Domaine : `CoverageReport`, `ApiCoverageMatrix`, `MutationReport`, `FlakinessReport`, `QualityGate`, `GapList`.

Ports in : `MeasureCoverageUseCase`, `RunMutationTestingUseCase`, `EvaluateQualityGatesUseCase`.

Ports out : `CoverageTool`, `MutationTool`, `ExecutionQuery`, `MetricsRepository`.

Adapters : JaCoCo, PIT, JPA.

Événement : `QualityMeasured` (avec la liste des lacunes).

Attention : PIT est coûteux : limité aux classes ciblées, déclenché en dernier.

6.12 `failure-analysis`

Rôle : comprendre **pourquoi un test échoue**. Il résout le problème de l'oracle : un échec peut venir du test, de l'environnement ou d'un vrai bug.

Responsabilités

- Classer chaque échec en `FailureCategory` :
  1. `TEST_ERROR` (compilation, import, mock, assertion mal formée) → envoyé à `repair-loop` ;
  2. `ENVIRONMENT_OR_FLAKY` (timeout, port occupé, données résiduelles) → relance ou quarantaine ;
  3. `SUSPECTED_APP_BUG` (test cohérent avec la spec, code incohérent) → **signalé, jamais « réparé »**.
- **Règles déterministes d'abord** (type d'exception, code HTTP vs spec, erreur de compilation), puis LLM pour les cas ambigus, avec la spec comme référence.
- Calculer un **niveau de confiance** par classification.
- Regrouper les échecs de même cause racine.
- Pour un bug probable : produire une **fiche** (requête, réponse obtenue, réponse attendue selon la spec, extrait de code suspect).

Domaine : `Failure`, `RootCause`, `SuspectedDefect` (utilise `FailureCategory` et `Confidence` du kernel).

Ports in : `AnalyzeFailuresUseCase`.

Ports out : `LlmClient`, `ApiCatalogQuery`, `SourceUnitQuery`, `FailureRepository`.

Événements : `FailuresClassified`, `DefectSuspected`.

6.13 `repair-loop`

Rôle : corriger automatiquement les tests classés `TEST_ERROR`.

Responsabilités

- Recevoir les échecs avec leur contexte (code du test, message d'erreur, code de la cible).
- Construire un **prompt de correction** ciblé (« ce test ne compile pas pour telle raison, voici les signatures réelles »).
- Demander la régénération au bon module de génération (unitaire, intégration, API).
- Relancer l'exécution et **itérer** jusqu'à N tentatives.

- Détecter la **stagnation** (même erreur répétée, régression) et arrêter tôt.
- **Interdire** les fausses corrections : supprimer l'assertion, affaiblir le test jusqu'à ce qu'il passe, commenter le test.
- En dernier recours : statut `DISCARDED` avec la raison, pour le rapport.

Domaine : `RepairAttempt`, `RepairPolicy` (max de tentatives, règles interdites), `RepairOutcome`.

Ports in : `RepairTestsUseCase`.

Ports out : `TestRegenerator` (appelle les ports in des modules de génération), `ExecutionGateway`, `RepairHistoryRepository`.

Événements : `TestRepaired`, `TestDiscarded`.

6.14 `workflow-orchestrator`

Rôle : le **chef d'orchestre** : enchaîne les étapes d'un run, gère l'état, le parallélisme et les reprises.

Responsabilités

- Modéliser un run comme **machine à états** : `CREATED → ANALYZING → PLANNING → PROVISIONING → GENERATING → EXECUTING → REPAIRING → MEASURING → REPORTING → COMPLETED` (ou `FAILED` / `CANCELLED`).
- Réagir aux **événements** des autres modules pour passer à l'étape suivante.
- **Paralléliser** par unité/endpoint avec limite de concurrence.
- Gérer **annulation, pause, reprise** après crash (état persisté).
- Appliquer les **limites globales** : durée maximale, budget LLM, nombre de passes.
- Décider de la **boucle** : relancer une passe ciblée si des lacunes persistent et que le budget le permet.
- Exposer la **progression** en temps réel (CLI et dashboard).

Domaine : `TestRun`, `RunState`, `RunConfiguration`, `RunStep`, `RunProgress`.

Ports in : `StartRunUseCase`, `CancelRunUseCase`, `ResumeRunUseCase`, `GetRunStatusQuery`.

Ports out : les ports in de tous les autres modules (via interfaces dédiées), `RunRepository`, `ProgressPublisher`.

Adapters : JPA, événements Spring, SSE/WebSocket.

Événements : `RunStarted`, `RunCompleted`, `RunFailed`.

Attention : seul module autorisé à connaître presque tous les autres.

6.15 `reporting`

Rôle : transformer les données du run en **rapport final** lisible.

Responsabilités

- Agréger : résultats d'exécution, couverture, mutation, échecs classés, bugs probables, coûts LLM.
- Produire les sections détaillées au §12.
- Générer plusieurs **formats** : HTML interactif, Markdown (description de PR), JSON (CI), PDF.
- Produire l'**export Postman/Newman**.
- Calculer des **tendances** entre runs.
- Archiver et rendre consultables les rapports.

Domaine : `RunReport`, `ReportSection`, `Finding`, `ReportFormat`.

Ports in : `GenerateReportUseCase`, `GetReportQuery`, `CompareRunsQuery`.

Ports out : requêtes vers les modules de données, `ReportRenderer`, `ReportStorage`.

Adapters : Thymeleaf (HTML), Markdown, JSON, OpenHTMLtoPDF, stockage fichier/S3.

Événement : `ReportGenerated`.

6.16 `vcs-integration`

Rôle : dialoguer avec **Git et les plateformes** (GitHub, GitLab).

Responsabilités

- **Cloner** le dépôt cible (branche/commit voulus).
- Calculer le **diff** pour les runs sur Pull Request.
- Créer une **branche dédiée** (ex. `qagent/tests-<runId>`) et **commiter** les tests retenus.
- Ouvrir une **Pull Request** avec le rapport Markdown en description.
- Publier un **statut/commentaire** sur la PR (quality gates passés ou non).
- Gérer l'authentification (tokens, GitHub App) avec permissions minimales.
- Mode **sans écriture** possible (rapport seul).

Domaine : `RepositoryRef`, `ChangeSet`, `CommitSpec`, `PullRequestSpec`.

Ports in : `CloneRepositoryUseCase`, `PublishTestsUseCase`, `GetChangedFilesQuery`.

Ports out : `GitClient`, `HostingPlatform`.

Adapters : JGit, API GitHub, API GitLab.

Événements : `RepositoryCloned`, `PullRequestOpened`.

6.17 `bootstrap`

Rôle : assemblage : l'application Spring Boot exécutable qui branche les ports sur leurs adapters et expose l'agent.

Responsabilités

- Classe `main`, configuration globale, profils (dev, prod), propriétés externalisées.
- **Interfaces d'entrée** : API REST (`/api/v1/runs`, `/reports`, `/projects`, documentée OpenAPI), **CLI** (Picocli), dashboard web simple.
- **Sécurité de l'agent** : authentification, rôles (admin, dev, lecteur), gestion des secrets.
- **Observabilité** : Actuator, Micrometer, logs structurés, traçage d'un run de bout en bout.
- Câblage des beans, `ApplicationModules.verify()` au démarrage, tests ArchUnit globaux.

Aucune logique métier ici. Si une règle apparaît dans `bootstrap`, elle est mal placée.

---

7. Règles de dépendance entre modules

Module	Peut dépendre de
`shared-kernel`	rien
`project-analysis`, `llm-gateway`, `environment-provisioning`, `vcs-integration`	`shared-kernel`
`api-discovery`	+ `project-analysis` (port in)
`test-planning`	+ `project-analysis`, `api-discovery`
`unit-/integration-/api-test-generation`	+ `llm-gateway`, `test-planning` (api-test-generation : + `api-discovery`)
`test-execution`	+ `environment-provisioning`
`quality-metrics`	+ `test-execution`, `api-discovery`
`failure-analysis`	+ `test-execution`, `api-discovery`, `project-analysis`, `llm-gateway`
`repair-loop`	+ `failure-analysis`, modules de génération, `test-execution`
`reporting`	+ lecture des modules de données (ports in de requête)
`workflow-orchestrator`	tous (via leurs ports in)
`bootstrap`	tous (assemblage)

Déclaration dans chaque module avec `@ApplicationModule(allowedDependencies = {...})`. Toute violation fait échouer le build. **Pas de dépendance cyclique** : les retours (ex. `quality-metrics` vers `test-planning`) passent par des **événements**, jamais par un appel direct.

---

8. Catalogue des événements

Tous implémentent `DomainEvent` (kernel) et portent un `runId`.

Événement	Émis par	Consommé par
`RunStarted`	workflow-orchestrator	project-analysis
`ProjectAnalyzed`	project-analysis	api-discovery, test-planning
`ApiDiscovered`	api-discovery	test-planning
`TestPlanCreated`	test-planning	modules de génération, workflow-orchestrator
`EnvironmentReady`	environment-provisioning	test-execution
`UnitTestsGenerated` / `IntegrationTestsGenerated` / `ApiScenariosGenerated`	modules de génération	test-execution
`TestsExecuted`	test-execution	failure-analysis, quality-metrics
`FailuresClassified`	failure-analysis	repair-loop, reporting
`DefectSuspected`	failure-analysis	reporting
`TestRepaired` / `TestDiscarded`	repair-loop	test-execution, reporting
`QualityMeasured`	quality-metrics	test-planning (passe ciblée), workflow-orchestrator, reporting
`ReportGenerated`	reporting	vcs-integration, workflow-orchestrator
`PullRequestOpened`	vcs-integration	workflow-orchestrator
`RunCompleted` / `RunFailed`	workflow-orchestrator	environment-provisioning (nettoyage), bootstrap

9. Pipeline d'un run

```
Demande (projet, périmètre, niveaux, environnement)
 → Clone / lecture du projet
 → Analyse du code + découverte des API (OpenAPI / Spring / Postman)
 → Planification (matrice de tests par niveau)
 → Provisionnement de l'environnement (base, dépendances mockées, application
démarrée)
 → [en parallèle, par unité ou endpoint]
      Génération → Compilation → Exécution
      └─ échec → Analyse de l'échec → Réparation (max N) ou signalement
 → Couverture code + couverture API → Mutation testing
 → Lacunes ? → passe ciblée supplémentaire (max M)
 → Rapport final → (option) commit + Pull Request
 → Destruction de l'environnement
```

---

10. Le problème de l'oracle de test

Un LLM peut écrire un test qui passe en **recopiant le comportement actuel**, y compris un bug. Parades :

1. **Classer chaque échec** (`failure-analysis`) : test faux, environnement/flaky, ou bug probable de l'application (voir §6.12).
2. **Assertions issues de la spécification** (OpenAPI, Javadoc, `@Valid`, règles métier), pas des réponses observées.
3. **Mutation testing** pour rejeter les tests sans valeur.
4. **Interdire les fausses réparations** (supprimer/affaiblir une assertion).
5. **Validation humaine optionnelle** avant commit.

Un test cohérent avec la spec qui échoue est un **résultat**, pas un problème à masquer.

---

11. Environnements, données et sécurité

Environnements et données

- Environnement **isolé et jetable** (Testcontainers/docker-compose), dépendances externes remplacées par WireMock.
- **Jamais de run sur la production** : liste blanche d'URL définie par l'admin.
- **Garde-fous** sur les opérations destructrices (DELETE, reset) : uniquement sur environnements « jetables ».
- **Authentification** : profils déclarés (admin, utilisateur, anonyme) ; l'agent obtient les tokens et teste aussi les cas 401/403.
- **Données** : factories, nettoyage après chaque scénario, aucune donnée réelle.
- **Secrets** : variables d'environnement ou coffre (Vault), jamais envoyés au LLM.

Sécurité de l'agent

- Le code analysé est **non fiable** (risque de *prompt injection* dans les commentaires) : il est isolé dans les prompts et jamais traité comme instruction.
- Exécution des tests générés **uniquement en sandbox** : limites CPU/RAM/temps, réseau restreint.
- Filtrage des secrets avant tout envoi à un LLM ; **mode 100 % local** pour les projets sensibles.
- **Journal d'audit** des appels LLM et des actions exécutées.
- Contrôle d'accès à l'API de l'agent (rôles, tokens).

---

12. Rapport final

1. **Résumé exécutif** : couverture avant/après, nombre de tests par niveau, taux de succès, durée, coût LLM.
2. **Couverture API** : matrice endpoint × méthode × code de statut (testé / non testé).
3. **Couverture de code** par module et classe, **mutation score**, mutants survivants.
4. **Bugs probables** : requête, réponse obtenue, réponse attendue selon la spec.
5. **Tests rejetés ou instables**, avec la raison.
6. **Zones non testables** et suggestions de refactoring.
7. **Écarts spec / implémentation** (champs manquants, codes non documentés).
8. **Traçabilité** : modèle LLM, version de l'agent, configuration.

Formats : HTML interactif, Markdown (PR), JSON (CI), PDF, plus collection Postman exportée.

---

13. Stack technique

Besoin	Choix
Langage / framework	Java 21, Spring Boot (dernière version stable, à vérifier au démarrage)
Build / modularité	Maven multi-module, Spring Modulith, ArchUnit
Analyse du code	JavaParser
API	swagger-parser / springdoc, import Postman (JSON)
LLM	Spring AI ou LangChain4j ; Ollama en local, cloud en option
Tests générés	JUnit 5, Mockito, AssertJ, REST Assured, Spring Boot Test
Environnement	Testcontainers, WireMock, Docker
Qualité	JaCoCo, PIT
Persistance	PostgreSQL + Flyway (H2 en dev)
Rapports	Thymeleaf + OpenHTMLtoPDF
Git / CI	JGit, API GitHub/GitLab, GitHub Actions / GitLab CI
Observabilité	Micrometer, Actuator, logs structurés
Interfaces	REST (OpenAPI), CLI (Picocli), dashboard web simple

14. Structure du dépôt

```
qagent/
├── README.md                          (ce document)
├── pom.xml                            (parent : BOM, versions, plugins)
├── docs/
│   └── adr/                           décisions d'architecture (ADR-0001,
...)
├── shared-kernel/                     ← créé en premier (voir §4)
├── project-analysis/
├── api-discovery/
├── test-planning/
├── llm-gateway/
├── unit-test-generation/
├── integration-test-generation/
├── api-test-generation/
├── environment-provisioning/
├── test-execution/
├── failure-analysis/
├── repair-loop/
├── quality-metrics/
├── workflow-orchestrator/
├── reporting/
├── vcs-integration/
├── bootstrap/                         application Spring Boot exécutable
└── golden-projects/                   applications d'exemple avec bugs
injectés
```

---

15. Gouvernance et conventions

Définition de « terminé » pour un module

- [ ] Structure hexagonale respectée (`domain` / `application` / `adapter`).
- [ ] Ports in et out documentés (Javadoc).
- [ ] Tests ArchUnit au vert, `ApplicationModules.verify()` au vert.
- [ ] Domaine testé sans Spring ; use cases testés avec ports mockés ; adapters testés en intégration.
- [ ] Événements émis/consommés ajoutés au catalogue §8.
- [ ] Fiche du module à jour dans ce README.

Conventions

- **Nommage :** ports in = `XxxUseCase` / `XxxQuery` ; ports out = noms métier (`LlmClient`, `GitClient`), jamais le nom de la techno ; adapters = `<Techno><Port>` (ex. `OllamaLlmProvider`, `JGitClient`).
- **Package racine :** `com.qagent.<module>` (ex. `com.qagent.apitest`).
- **Événements :** nom au passé (`TestsExecuted`), immuables, `runId` obligatoire.

- **Erreurs :** exceptions dérivées de `QAgentException`, jamais d'exception de framework dans le domaine.
- **Décisions d'architecture :** toute décision structurante fait l'objet d'un **ADR** dans `docs/adr/`.
- **Modifier le shared-kernel :** suivre la règle SK-7 (§4.4).
- **Modifier ce README :** obligatoire dans la même PR que tout changement d'architecture.

---

16. Stratégie de test de QAgent lui-même

Couche	Approche
Domaine	Tests unitaires purs, sans Spring
Use cases	Ports mockés ; `FakeLlmProvider` déterministe
Adapters	Tests d'intégration (Testcontainers)
Architecture	ArchUnit + Spring Modulith à chaque build
Qualité de l'agent	**Golden projects** : applications Spring d'exemple avec bugs injectés ; on mesure à chaque version la qualité des tests générés et le taux de bugs détectés

---

17. Risques et parades

Risque	Parade
Tests qui figent un bug	Assertions basées sur la spec, classification des échecs
Couverture sans valeur	PIT, rejet des tests sans assertion
Hallucination d'API ou de classes	Contexte avec signatures réelles + boucle de compilation
Scénarios API incohérents (dépendances entre appels)	Graphe de dépendances entre endpoints, modèle de scénario validé avant génération
Dégâts sur un environnement réel	Liste blanche, environnements jetables, garde-fous DELETE
Gros projets (limite de contexte)	Découpage par unité/endpoint, résumés, cache
Coût et latence du LLM	Parallélisme borné, budget par run, modèle local
Tests flaky	Exécutions multiples, quarantaine
Dérive d'architecture	ArchUnit + Modulith en CI
Shared-kernel qui grossit	Règles SK-1 à SK-8, revue à chaque phase

18. Feuille de route

Phase	Contenu	Livrable	Durée indicative
0. Cadrage	Choix techniques, **shared-kernel**, squelette multi-module, CI, tests d'architecture, projet golden	Dépôt + pipeline	1-2 sem.
1. MVP unitaire	`project-analysis` + `unit-test-generation` + `llm-gateway` (Ollama) + `test-execution` + rapport Markdown via CLI	1 classe → tests qui passent	3-4 sem.
2. API	`api-discovery`, `api-test-generation`, `environment-provisioning`, authentification	Tests API d'un service complet	4-5 sem.
3. Boucle intelligente	`repair-loop`, `failure-analysis`, JaCoCo, couverture API, parallélisme, `workflow-orchestrator`	Détection de bugs + rapport HTML	3-4 sem.
4. Intégration et qualité	`integration-test-generation`, PIT, flaky, import/export Postman, `vcs-integration`, mode CI	Adoption par les équipes	4 sem.
5. Durcissement	Sandbox complet, sécurité de l'agent, observabilité, multi-LLM, dashboard	Version production	3-4 sem.

Total indicatif : **4 à 5 mois** pour une personne, moins en équipe. Le **MVP utile** (phases 0 à 3) tient en **3 à 4 mois**.

---

19. Exemples d'usage

bash

```bash
qagent run --project ./mon-app --levels unit,integration,api \
  --openapi ./openapi.yaml --env ephemeral --min-coverage 80 \
  --llm ollama:qwen2.5-coder
```

http

```
POST /api/v1/runs
{
  "repoUrl": "...",
  "levels": ["UNIT", "INTEGRATION", "API"],
  "targetEnvironment": "ephemeral",
  "postmanCollection": "collection.json",
  "maxRepairIterations": 3,
  "qualityGates": { "minCoverage": 80, "minApiCoverage": 100 }
}

GET /api/v1/runs/{id}/report?format=html
```

20. Décisions ouvertes

#	Question	Options	Décision
D1	LLM	Local uniquement / cloud aussi	À prendre
D2	Cible du MVP	Projets Spring Boot uniquement / toute API (via OpenAPI seul)	À prendre
D3	Distribution	CLI + CI d'abord / serveur centralisé avec dashboard	À prendre
D4	Build des projets analysés	Maven seul / Gradle aussi	À prendre
D5	Validation	100 % automatique / revue humaine avant commit	À prendre
D6	Version de Spring Boot et bibliothèque LLM (Spring AI vs LangChain4j)	À vérifier au démarrage	À prendre

Chaque décision prise est consignée dans un ADR et reportée dans ce tableau.

21. Glossaire

Terme	Définition
Run	Une exécution complète de l'agent sur un projet
Cible (target)	Unité à tester : classe, méthode ou endpoint
Port in / port out	Interface exposée par un module / interface dont un module a besoin
Adapter	Implémentation technique d'un port (JPA, Git, LLM, Docker…)
Shared kernel	Vocabulaire minimal partagé par tous les modules (§4)
Oracle de test	Source de vérité qui dit quel résultat est « correct »
Quality gate	Seuil de qualité à respecter (ex. couverture ⩾ 80 %)
Mutation score	Part des fautes injectées que les tests détectent
Mutant survivant	Faute injectée que aucun test n'a détectée
Flaky	Test dont le résultat varie sans changement de code
Golden project	Application d'exemple à bugs connus, pour évaluer l'agent
ADR	Architecture Decision Record : trace écrite d'une décision
Sandbox	Environnement d'exécution isolé et limité